JARVIS: Personal Voice Assistant for Mac

Description, architecture, tools and build plan · Prepared for Aayush Kamble · 10 November 2026

1. Overview

JARVIS is a personal assistant that runs on your MacBook. You say a wake word, speak naturally, and it answers by voice and carries out actions for you: opening and closing apps, searching the web, reading email, controlling smart-home devices, remembering things, and more.

It has three layers:

- **Ears and mouth:** wake word, voice activity detection, speech-to-text (STT) and text-to-speech (TTS).
- **Brain:** a large language model (LLM) that understands the request and decides which tool to call.
- **Hands:** tools that act on the Mac, the web and your accounts.

"Can do anything" is achieved by growing the tool library over time. Each tool is a small, tested Python function, and the brain chooses between them. The first version does a few things well, and every later phase adds more.

Why a native Mac program and not a website: a web page runs inside the browser sandbox. It can use the microphone and play audio, but it cannot open or close other apps, run commands or control windows. So the core agent runs natively on the Mac. A web dashboard can be added later as an optional face for it.

2. Goals and success metrics

Goals

- Talk to JARVIS hands-free and get a spoken reply that feels instant.
- Execute real actions on the Mac and in online services.
- Remember preferences and facts across days.
- Stay safe: never do something destructive without confirmation.
- Double as a portfolio project that shows measured performance, not just a demo.

Non-goals (for now)

- Running on a phone as a standalone app.
- Multi-user support or voice identification.
- Fully offline operation at cloud quality. Offline mode is a fallback, not the main path.

Success metrics

Metric	Target
Time from end of speech to first audio reply	under 1.0 s (stretch), under 1.5 s (acceptable)
Correct tool and arguments on a 50-command test set	90% or more
Wake word false activations	fewer than 2 per day
Wake word misses	fewer than 1 in 10 attempts
Destructive actions run without confirmation	zero
Crash-free uptime over a day	99% or better

3. Architecture

```
Mic → Wake word → VAD → Speech-to-text (streaming)
                                ↓
          Router: simple command, or needs the LLM?
                ↓                          ↓
        Fast path (rules)      Brain: LLM + tool calling ↔ Memory
                ↓                          ↓
             Tool executor (permission and risk check)
                ↓
   Mac control · Web · Email/Calendar · Smart home · Vision
                ↓
           Text-to-speech (streaming) → Speaker
```

Components

Component	Responsibility
Audio input	Captures the microphone stream and applies a noise gate
Wake word	Listens for "Jarvis" locally and cheaply, always on
VAD	Detects when you start and stop speaking
STT	Turns speech into text as you speak (streaming)
Router	Sends trivial commands (volume, timers) down a fast path and everything else to the LLM
Brain	LLM with a list of tools; decides what to call and what to say
Tool executor	Validates arguments, checks the risk level, asks for confirmation if needed, runs the tool, logs the result
Memory	Stores facts, preferences and searchable notes
TTS	Speaks the reply sentence by sentence as the LLM streams it
Orchestrator	Runs the state machine: idle, listening, thinking, acting, speaking

Life of one request

1. The wake word fires and JARVIS plays a short chime.
2. VAD marks the start of speech and STT streams text.
3. VAD detects silence, the transcript is finalized and sent to the router.
4. The brain receives the transcript, recent conversation, relevant memories and the tool list.
5. If a tool is needed, the brain returns a tool call. The executor checks risk, runs it and returns the result to the brain.
6. The brain streams its final answer. Each finished sentence goes straight to TTS so audio starts early.

7. If you speak during the reply (barge-in), TTS stops and JARVIS listens again.

Fast path and slow path

Some commands do not need an LLM. "Volume 40," "mute," "set a timer for ten minutes" and "open Spotify" can be matched with simple rules or a small local classifier. This makes them near-instant and costs nothing. Everything ambiguous goes to the LLM.

Background tasks

Long jobs ("research X and summarize it") should not block the conversation. JARVIS says "On it," runs the job in a background worker, and announces the result when it finishes.

4. Feature catalog

Tier 1: core (build first)

- Push-to-talk and wake word activation.
- Spoken conversation with short-term context.
- Open and close applications, open URLs and Gmail, adjust volume and brightness.
- Web search with spoken summaries.
- Timers, reminders and quick notes.

Tier 2: daily assistant

- Read and summarize unread email, draft replies (send only after confirmation).
- Read and create calendar events.
- Control Spotify (Premium is required for playback control through the API).
- File search and opening ("open my resume").
- "Remember that..." and "what did I tell you about..." using long-term memory.
- Morning briefing: date, weather, calendar, top news and reminders.
- Browser automation: log in, click, fill forms, scrape pages (Playwright).

Tier 3: advanced

- **Screen awareness:** "what's on my screen?", "explain this error."
- **Question answering over your own documents** (RAG over notes, PDFs and project folders).

- **Smart home and IoT:** relays, sensors and lights through MQTT and ESP boards.
- **Proactive alerts:** meeting reminders, deadlines, threshold alerts.
- **Phone access** through a Telegram bot or a secured web app.
- **GUI control** of apps that have no API, using computer-use style screenshot and click loops.
- **Dashboard (HUD):** a React page on localhost showing the waveform, transcript, action log and status.

5. Tools and technology

5.1 Stack

Layer	Primary choice	Cost	Alternative
Language	Python 3.11+ with asyncio	Free	
Editor and VCS	VS Code, Git, GitHub	Free	
Wake word	openWakeWord	Free, open source	Custom-trained wake word
VAD	Silero VAD	Free	WebRTC VAD
STT (cloud)	Deepgram streaming	$200 free credit, then pay per minute	OpenAI transcription
STT (local)	faster-whisper or whisper.cpp	Free	mlx-whisper on Apple Silicon
LLM brain	Claude API: Haiku 5.5 for speed, Sonnet 5.5 for harder tasks	Pay per token	Ollama with a small local model
TTS (day 1)	macOS `say`	Free	
TTS (local)	Kokoro or Piper	Free	
TTS (cloud)	ElevenLabs or Cartesia	Limited free tiers, then paid	Deepgram Aura

Layer	Primary choice	Cost	Alternative
Mac control	`open`, `osascript` (AppleScript), Shortcuts	Free, built in	`pyautogui`
Browser automation	Playwright	Free	
Web search	DuckDuckGo search library to start	Free	Tavily or Brave Search API
Memory	SQLite plus Chroma or sqlite-vec	Free	FAISS
Integrations	Gmail and Calendar APIs, MCP servers	Free within quota	Zapier or n8n
Smart home	Mosquitto (MQTT) plus ESP32 or ESP8266	Free	Home Assistant
Tracing and evals	Langfuse (self-hosted) or MLflow	Free	
Dashboard	React on localhost, WebSocket to the agent	Free	
Remote access	Telegram Bot API, Tailscale	Free tiers	

5.2 Pricing notes (checked November 2026; verify before relying on them)

- **Wake word:** Picovoice Porcupine's free tier was discontinued on 30 June 2026, so openWakeWord is the free route.
- **Claude API:** there is no ongoing free tier, only a small test credit for new accounts. Haiku 5.5 costs $0.10 per million input tokens and $0.50 per million output tokens for prompts up to 100K tokens.
- **Deepgram:** new accounts receive $200 of free credit with no credit card required. Sources disagree on whether the credit expires, so confirm when you sign up.
- **Spotify:** the Web API is free, but playback control needs a Premium account.
- **Google APIs:** free for personal use, but an OAuth app in testing mode may force you to sign in again about once a week.

5.3 Do we need LangChain, n8n or an agent framework?

Not for the core.

Tool	Verdict
LangChain	Skip at first. The tool loop is about 50 lines with the Claude SDK, and writing it yourself teaches you how tool calling works. LangGraph can be added later for complex multi-step flows.
n8n or Zapier	Useful from Phase 4 for multi-app actions ("email this file, log it in a sheet, notify me"). Too slow for the real-time loop, so use it only for actions.
Agent SDKs	Useful later for long background research tasks.
MCP	Worth using. It is the standard way to plug in Gmail, Calendar, Drive and other tools without writing each integration.
Pipecat or LiveKit Agents	The right frameworks for real-time voice. Build a basic loop by hand first, then decide whether you need one.

5.4 Two build tracks

Track A: fully free. openWakeWord, faster-whisper, an Ollama model and Kokoro or Piper. It costs nothing, but expect slower replies, lower accuracy and weaker tool calling on a laptop without a GPU.

Track B: best quality, very low cost. Free wake word and VAD, Deepgram on its free credit for STT, Claude Haiku for the brain, and macOS `say` or Kokoro for TTS. Personal-use costs should stay in the range of a few dollars a month. As a rough estimate, one command with tool definitions costs a fraction of a cent, so about one dollar per thousand commands.

Recommendation: build Track B for quality, but keep the interfaces swappable so a local model can replace any cloud part.

6. Tool-calling design

6.1 The tool registry

Every capability is a Python function with a name, a description, a JSON schema for its arguments and a risk level. The brain sees the names, descriptions and schemas, and returns a call. The executor validates the arguments before running anything.

```
import subprocess

def open_app(name: str) -> str:
    subprocess.run(["open", "-a", name], check=True)
    return f"Opened {name}"

TOOLS = {
    "open_app": {
        "fn": open_app,
        "risk": "safe",
        "schema": {
            "name": "open_app",
            "description": "Open a macOS application by name",
            "input_schema": {
                "type": "object",
                "properties": {"name": {"type": "string"}},
                "required": ["name"],
            },
        },
    },
}
```

6.2 Risk levels

Level	Meaning	Behavior
safe	Read-only or trivially reversible (open an app, read the weather)	Runs immediately
confirm	Has side effects (send email, create event, delete or move files, run a shell command)	JARVIS states what it will do and waits for a spoken "yes"

Level	Meaning	Behavior
blocked	Never allowed (format disk, bulk delete, payments)	Refused

6.3 Initial tool catalog

Tool	What it does	How
open_app / quit_app	Opens or quits an app	`open -a`, AppleScript `quit`
open_url	Opens a page, including Gmail	`open <url>`
set_volume / set_brightness	System controls	AppleScript, `osascript`
web_search	Searches and returns snippets for the LLM to summarize	Search API
fetch_page	Reads a page for details	HTTP fetch plus text extraction
set_timer / add_reminder	Timers and reminders	Local scheduler, optional Reminders app
take_note / search_notes	Stores and retrieves notes	SQLite plus vector search
find_file / open_file	Locates and opens files	`mdfind` (Spotlight)
read_email / draft_email / send_email	Email (send is confirm)	Gmail API or MCP
list_events / create_event	Calendar (create is confirm)	Calendar API or MCP
play_music / pause / next	Spotify control	Spotify Web API (Premium)
run_shell	Runs a command	Allowlist plus confirm
browser_task	Navigates, clicks, fills forms	Playwright
screen_describe	Captures the screen and asks a multimodal model about it	`screencapture` plus vision model
home_control	Switches a relay or reads a sensor	MQTT publish and subscribe

Tool	What it does	How
remember / recall	Long-term memory	Memory store

7. Mac integration

- **Permissions (System Settings, Privacy and Security):** Microphone, Accessibility, Automation (per app) and Screen Recording (only for vision features). These are one-time prompts.
- **Run at login:** a `launchd` agent in `~/Library/LaunchAgents`, or a small menu bar app, so JARVIS is always listening.
- **Run the agent natively.** Docker on macOS runs inside a Linux VM and cannot reach the microphone or control Mac apps. Use Docker only for supporting services such as Mosquitto, n8n and Langfuse.
- **Headphones first.** Without echo cancellation, JARVIS can hear its own voice through the speakers and interrupt itself. Test barge-in with headphones, then tune for speakers.
- **Secrets:** API keys and tokens in a `.env` file or the macOS Keychain, never in source code.

8. Memory design

Memory type	Content	Storage
Short-term	Last several turns of conversation	In process, trimmed by token count
Facts and preferences	"My manager's name is…", "I prefer brief answers"	SQLite key-value table
Notes and documents	Notes, PDFs, project docs	Vector store with embeddings
Action log	Every tool call, arguments, result, timestamp	SQLite (also feeds the audit trail and evals)

Only store what you tell JARVIS to remember. Show and delete memories on request with commands like "what do you remember about me?" and "forget that."

9. Safety and security

- The risk-level system in section 6.2 is enforced in the executor, not left to the LLM's judgment.
- Shell commands run only from an allowlist, or after a spoken confirmation.
- Treat anything JARVIS reads from the web, email or files as untrusted data. Text inside a web page or email must never be allowed to trigger actions. This is the main prompt-injection risk for any assistant with tools.
- Bind any local server (dashboard, WebSocket) to `localhost`. If you expose it remotely, use Tailscale or authentication, never an open port.
- Keep an audit log of every action and let yourself review it.
- Add a kill phrase ("Jarvis, stop everything") and a hotkey that halts all running tasks.

10. Project structure

```
jarvis/
├── main.py                 # entry point, starts the orchestrator
├── config.py               # settings, loads .env
├── audio/
│   ├── wake.py             # openWakeWord
│   ├── vad.py              # Silero VAD
│   ├── stt.py              # Deepgram or faster-whisper
│   └── tts.py              # say / Kokoro / cloud
├── brain/
│   ├── llm.py              # Claude client, streaming, tool loop
│   ├── router.py           # fast path vs LLM path
│   └── prompts.py          # system prompt
├── tools/
│   ├── registry.py         # tool schemas, risk levels, executor
│   ├── mac.py              # apps, volume, files
│   ├── web.py              # search, fetch, Playwright
│   ├── google.py           # Gmail, Calendar
│   ├── music.py            # Spotify
│   ├── home.py             # MQTT
│   └── vision.py           # screen capture
├── memory/
│   ├── store.py            # SQLite
│   └── vectors.py          # embeddings and search
├── ui/                     # optional React HUD
├── evals/
```

```
|   ├── commands.jsonl       # test set
|   └── run_evals.py
├── tests/
└── README.md
```

11. Roadmap

Estimated at 8 to 10 weeks with roughly 8 to 10 hours per week alongside college. Each phase ends with something you can use and a check that it works.

Phase	When	What you build	Done when
0. Setup	Days 1 to 2	Repo, virtual environment, API keys, microphone test, macOS permissions	A script records 5 seconds of audio and plays it back
1. Voice loop MVP	Week 1	Push-to-talk, STT, LLM, TTS	You speak, and a spoken answer arrives in under 2 s
2. Real-time feel	Week 2	Wake word, VAD end-of-speech, streaming STT and TTS, barge-in	Time to first audio is about 1 s and you can interrupt it
3. Command execution	Weeks 3 to 4	Tool registry, executor, risk levels; app control, URLs, volume, web search, timers, notes	90% of your 50-command test set passes
4. Integrations	Week 5	Gmail, Calendar, Spotify through APIs or MCP; optional n8n for multi-step flows	"Summarize my unread email" and "what's on my calendar today" work
5. Memory and knowledge	Week 6	Fact store, vector search, document Q&A	JARVIS recalls a fact stated days earlier and answers from your notes
6. IoT and vision	Week 7	MQTT to an ESP relay, screen capture and description	"Turn on the lamp" and "explain this error" work

Phase	When	What you build	Done when
7. Proactive and polish	Week 8	Morning brief, reminders, alerts, launchd autostart	It starts at login and delivers the brief unprompted
8. Dashboard and MLOps	Weeks 9 to 10	React HUD, tracing, latency dashboard, CI, README and demo video	A recorded demo plus a metrics report

12. Testing and MLOps

This is the part that turns a demo into a portfolio project.

- **Command test set:** 50 or more spoken-style commands with the expected tool and arguments (for example, "open Spotify" maps to `open_app(name="Spotify")`). Run it after every change and track accuracy.
- **Latency tracing:** log timestamps for wake, end of speech, STT final, LLM first token, first TTS audio. Report p50 and p95 for each stage.
- **Tracing tool:** Langfuse or MLflow to store each request, tool call and result for debugging.
- **Unit tests** for every tool, with fake inputs and no real side effects.
- **CI:** GitHub Actions running the unit tests and the text-only evals (no audio) on every push.
- **Regression log:** keep failed commands in the test set so they never fail again.
- **Honest reporting:** publish your measured numbers in the README, including the failures.

13. Risks and mitigations

Risk	Mitigation
Latency feels slow	Stream every stage, use Haiku for most requests, keep the fast path for simple commands, speak the first sentence early
Wake word false triggers	Tune the threshold, add a confirmation chime, log false triggers and retrain or adjust

Risk	Mitigation
LLM picks the wrong tool	Clear tool descriptions, few and distinct tools, evals, and a confirmation step for risky actions
Prompt injection from web or email content	Treat external content as data, never as instructions; require confirmation for any action that follows reading external content
API costs grow	Use Haiku by default, cache the system prompt and tool list, set a monthly spending limit
Google OAuth hassles	Use testing mode with your own account, store refresh tokens securely, or use MCP connectors
Scope creep	Finish each phase's acceptance check before starting the next
Provider pricing changes	Keep STT, TTS and LLM behind small interfaces so any can be swapped

14. Immediate next steps

1. Choose a track: A (fully free) or B (cloud quality at low cost).
2. Create the GitHub repo and the project structure above.
3. Set up a Python virtual environment and confirm microphone access.
4. Create a Claude API account and a Deepgram account (Track B) and put the keys in `.env`.
5. Build Phase 1: the push-to-talk voice loop.
6. Write your first 20 test commands now, even before the tools exist. They define what "working" means.